

Day 1 — Practice Questions

MongoDB Foundations & CRUD — on MongoDB Atlas

Work through these alongside each topic as it's taught. Everything runs against sample_mflix, the free movie database built into MongoDB Atlas — you'll be querying real movies, not made-up records. Use mongosh for typed commands and the Atlas Data Explorer (Browse Collections) to double-check visually.

Before you insert or delete anything: tag every document YOU add with custom: true. It makes your own test data easy to find and clean up later without ever touching the real sample_mflix data.

Module 1 — Welcome, Orientation & Atlas Setup

Get your own Atlas cluster live, load the sample data, and connect mongosh to it.

1. Create your own free Atlas account and an M0 cluster.
2. Create a database user (note the password) and set Network Access to 'Allow from Anywhere'.
3. Load the Sample Dataset on your cluster and wait for it to finish.
4. Open Browse Collections (the Data Explorer) and confirm sample_mflix appears with multiple collections inside it.
5. Install mongosh, then connect it to your cluster using the mongodb+srv string from Atlas -> Connect -> Shell.
6. Confirm your mongosh prompt shows 'Atlas ... [primary]', then run show dbs and confirm sample_mflix is listed.
7. Confirm both the Data Explorer shows sample_mflix AND mongosh is connected with show dbs working.

Module 2 — NoSQL & the Document Model

Design a document of your own — no database needed yet, just paper or a text file.

1. Design a document for a 'library_book' with at least 6 fields.
2. Include at least one nested object (e.g. publisher: { name, city }).
3. Include at least one array (e.g. tags: ['fiction', 'bestseller']).
4. Be ready to read yours out loud.

Module 3 — Exploring the sample_mflix Database

Find your way around data that's already there — nothing to build yet.

1. Switch to sample_mflix in mongosh with use sample_mflix.
2. Run show collections and list everything you see.
3. Run countDocuments() on both movies and comments, and write down the two numbers.
4. In the Data Explorer, open the movies collection and click into 3 documents you don't recognize — notice how their fields aren't all identical.

Module 4 — Inserting Documents

Add your own picks to the real movies collection. Tag every one with custom: true.

1. Using the UI (INSERT DOCUMENT button), insert one movie of your own choosing — include title, year, genres, runtime, cast, plot, and custom: true.
2. Using mongosh and insertMany(), insert 3 more movies (all tagged custom: true) — mix up genres and years.
3. Run `db.movies.countDocuments({ custom: true })` and confirm the number matches how many you inserted (should be 4 total).
4. Refresh the Data Explorer, filter by `{ custom: true }`, and confirm your documents appear there too.

Module 5 — Querying Documents (find & operators)

Write and run a query for each — type every one yourself, don't copy blindly.

1. All movies in the 'Comedy' genre.
2. All movies with `imdb.rating` greater than or equal to 8.5.
3. All movies released in 1994 OR 1999 (use `$in` on year).
4. All movies that are NOT rated 'R' (use `$ne` on rated).
5. All movies where the cast array includes an actor of your choice.
6. All movies in 'Comedy' AND released after 2010 (implicit AND).
7. Bonus: use `countDocuments()` to report how many movies have `imdb.rating` above 9.

Module 6 — Updating & Deleting Documents

Only ADD fields to real movies — never overwrite an existing one. Only bulk-delete your own custom: true data.

1. Using `updateOne()`, add a field like `watched: true` to one real movie of your choice (match it by title).
2. Using `$inc`, increase the runtime of one of YOUR OWN custom: true movies by 10 — never touch a real movie's runtime.
3. Using `updateMany()`, add a field `recommended_by_class: true` to every movie in a genre of your choice.
4. Run `find()` to confirm the field appears on the right movies.
5. Using `deleteOne()`, remove ONE of your own custom: true movies — but run `findOne()` with that filter FIRST.
6. Using `deleteMany({ custom: true })`, clean up ALL of your own test movies at once — `find()` first, then confirm `countDocuments({ custom: true })` returns 0 afterward.

Module 7 — Projection, Sort, Limit & Skip

Shape how results come back, not which documents match.

1. Return only title and year for movies in a genre of your choice (hide `_id`).
2. Return the top 5 movies by `imdb.rating`, highest first.
3. Return the 5 oldest movies in the whole collection (sort by year ascending).
4. Find 'page 2' of Comedy movies sorted by year descending, 5 per page (skip 5, limit 5).

5. Do the top-5-by-rating task again, but in the Data Explorer Options panel instead of the shell — confirm you get the same movies.

Module 8 — Schema Design: Embedding vs Referencing

sample_mflix already contains a real example of each pattern — find them.

1. Pick any real movie and run `findOne()` projecting only title, genres, and cast — confirm both are arrays embedded right in the document.
2. Run `db.comments.findOne()` and identify its `movie_id` field.
3. Use that `movie_id` in a second query against movies to find which movie the comment belongs to.
4. In your own words, write down why comments are referenced rather than embedded inside each movie.

Module 9 — Capstone Lab 1

Everything from today, in one exercise. Use mongosh, and the Data Explorer to double-check.

1. Confirm you're connected to `sample_mflix` and note roughly how many documents are in `movies` and in `comments` (`countDocuments()`).
2. Insert 2 of your own favourite movies into `movies`, both tagged `custom: true`.
3. Write and run 5 DIFFERENT `find()` queries against `movies`, each using a different feature: one `$gt/$gte`, one `$in`, one `$or`, one array match (genres or cast), one implicit AND.
4. Update 2 documents — one REAL movie with `updateOne()` + `$set` (add a field, never overwrite an existing one), one genre-wide change with `updateMany()`.
5. Clean up your own test data: `deleteMany({ custom: true })` — `find()` first, then confirm `countDocuments({ custom: true })` returns 0 afterward.
6. Run one projection query returning only 2-3 chosen fields (`hide _id`).
7. Run one query chaining `sort()`, `limit()`, and `skip()` together.
8. Look at one `comments` document, find its `movie_id`, and manually look up the movie it belongs to.
9. Cross-check any two of the above in the Data Explorer UI and confirm the results match the shell.

Finished early? Stretch question: write a query that finds movies where `imdb.rating` is above 8.5 AND the `genres` array includes 'Drama' plus at least one other genre of your choice.